Sharing Conflict Weights Without Partitioning: A Concurrent dom/wdeg Portfolio for Word-Level Crossword Filling

Luke K. Schreiber
Department of Computer Science
Princeton University
Princeton, NJ, USA
ls1881@princeton.edu

Abstract—Parallel CSP solvers typically take one of two forms: partition the search space across workers, or run an undivided portfolio of independent restarts. When CSP workers share information at all, prior work treats that sharing as a one-time input to partitioning, gathered once and then spent. This paper describes a different mechanism: a restart portfolio whose workers never partition the problem, but continuously and concurrently update a shared pool of dom/wdeg-style conflict weights throughout the search, with no manager and no synchronization phase. We implement it in xfill, a word-level CSP solver for crossword filling, and test it two ways. First, we establish that restart-portfolio search as a class occupies a genuinely different point in the design space than partition-based search, using orca-solver, an independently developed partition-based solver, as a concrete representative of the alternative. Restart portfolios exploit heavy-tailed search-runtime distributions: oversubscribing thread count past the physical core count improves xfill by roughly 65 times on one hard grid, while the partition-based solver improves only marginally and cannot solve it at all below 14 threads. On realistic-scale instances, xfill solves every one of 14 real, previously published 15x15 grids tested, and on the 11 where the partition-based solver also succeeds, xfill is faster on 9, by a geometric mean of about 107 times. Partitioning, conversely, gives genuine non-overlapping coverage, a real advantage for proving unsatisfiability. Second, and more directly on point, we isolate the shared-weight mechanism itself, holding the solver and grid fixed and toggling only the mechanism, and find a more nuanced result than the architectural comparison above: consistent improvement on a broad corpus of moderate-difficulty grids, but higher variance rather than a reliable win on especially hard instances. A negative or mixed result for one's own mechanism is still evidence, so we present it alongside the positive one rather than in place of it. We also report a negative result from the same codebase: five independently plausible letter-level branching heuristics produced numerically identical output on our hardest benchmark grid, because a domain-size precondition meant none of them ever executed, caught only by direct execution-path instrumentation and not by comparing benchmark scores.

Index Terms—constraint satisfaction, parallel search, restart portfolios, dom/wdeg heuristic, crossword generation

I. Introduction

Crossword filling – assigning a dictionary word to every slot in a grid so every crossing letter agrees – is a word-level CSP and a convenient search-research testbed: instances are human-legible, difficulty is controllable directly through grid geometry, and independently developed solvers already exist to compare against. Ginsberg's Dr.Fill [1] established crossword filling as a serious weighted-CSP application over a decade ago.

This paper is not about crossword filling. It uses crossword filling as a setting to answer one question about parallel constraint search: when multiple workers run concurrently, is there value in sharing soft heuristic statistics – information that biases search order without asserting anything logically – continuously, without ever partitioning the problem those workers are solving? Section II shows this combination does not exist in prior work; Section III implements it; Section V measures its effect, both in isolation and against the broader architectural backdrop that makes it worth measuring at all.

Contributions: (1) a concurrent, decentralized conflict-weight-sharing mechanism for restart-portfolio CSP search, positioned precisely against the closest prior work; (2) a reproducible four-solver testbench that separates two claims – restart-portfolio search as an architectural class versus this specific mechanism within it – rather than conflating them; and (3) a methodological negative result about confirming a heuristic under test actually executes before trusting a benchmark comparison of it.

II. Background and Related Work

A CSP assigns values to variables from finite domains subject to constraints; systematic backtracking assigns one variable at a time, propagating consistency and backtracking on a wipeout. The dom/wdeg heuristic [4] selects the variable with smallest domain-size-to-weighted-degree ratio, where a constraint's weight increases on every wipeout it causes. Randomized backtracking exhibits heavy-tailed runtime [2], [3]: restarting periodically eliminates most of the tail and is standard in SAT and CSP solvers alike.

Two dominant parallel strategies exist. Portfolio approaches run independent searches over the whole problem concurrently; SAT portfolios such as ManySAT [6] additionally share learned clauses between workers without partitioning. Partitioning approaches divide the search

space so each worker explores a disjoint region, giving non-overlapping coverage useful for proving unsatisfiability.

Three prior CSP results sit near, but not on, the point in this design space we describe. Grimes and Wallace [5] accumulate dom/wdeg-style weights from short restarted probes of a single sequential solver – never concurrent. Ehlers and Stuckey [7] parallelize a lazy-clause-generation solver by sharing nogoods and trail state between workers – concurrent, but sharing hard, logically derived facts, and requiring clause-learning machinery most CSP solvers lack. Yun and Epstein's SPREAD [8] is the closest: a manager races workers under a plain dom/wdeg solver, then averages their reported weights once each exhausts a budget, and uses the average to choose how to partition the problem for a subsequent phase. The authors state this directly: "the primary purpose of SPREAD's portfolio phase is to glean information to support search space splitting, not to solve P" [8]. A comprehensive 2018 survey [9] documents no CSP work sharing heuristic statistics continuously among concurrent, non-partitioning workers; its own taxonomy has no category for it.

The mechanism in Section III occupies the gap these leave: concurrent, unlike Grimes and Wallace; soft, unlike Ehlers and Stuckey; and never used to partition, unlike Yun and Epstein.

III. System Architecture

xfill parses a grid into slots (open runs ≥ 2 cells) and crossings (cell-level intersections). Its sequential core performs dom/wdeg-style branching – selecting the slot minimizing domain size over accumulated crossing weight – then propagates letter constraints via an AC-3-style queue; a domain wipeout increases the responsible crossing's weight.

SolveParallel runs N independent restart workers, each with its own seed and private, decaying crossing-weight table, plus one worker dedicated to a single uninterrupted exhaustive search (unlimited_budget) that guarantees eventual completion independent of restart variance. Any worker's sound negative result cancels the others.

Concurrent conflict-weight sharing. Alongside each worker's private table, SolveParallel maintains one array of plain atomic counters, one per crossing, shared by every restart worker except the dedicated exhaustive one. Every worker increments a crossing's counter on wipeout and reads every counter when scoring candidates. The counters are plain, order-independent atomic increments rather than a decaying scheme: a decaying normalizer depends on the exact sequence of prior updates and is race-free only with a single writer, while concurrent writers need an update rule that does not depend on order. The dedicated exhaustive worker is excluded because its entire value is one undisturbed trajectory; wiring it in measurably regressed the one grid in our corpus it uniquely solved. Two correctness properties were required: single-threaded calls stay byte-identical to the unshared baseline, gated on thread count > 1; and ThreadSanitizer, run against real multi-threaded solves, reports no data races, consistent with every update being independent of order. In earlier development-time testing across a broad corpus of moderate-difficulty grids (documented in docs/design.md), this mechanism cut solve time broadly – a finding we revisit against a harder, targeted ablation in Section V-G.

IV. A Negative Result: Branching on the Wrong Thing

This section is a methodological aside from the same codebase; we include it because it shaped how we validate every other result in this paper. Motivated by a 7×7 grid known to be exceptionally hard for restart-based search (Section V-B), we implemented five letter-level branching variants – two minimum-remaining-value scoring rules, a faithful reimplementation of a partition-based solver's own candidate scoring (read from its published source for reference), and a reversed scan order – reasoning that a smaller per-node branching factor should help on wide-open grids.

All five produced numerically identical node and backtrack counts – too strong a signal to report as a simple null result, so we checked further. A plain counter in the branch-selection code, tracking how often the large-domain branch these heuristics govern actually executed, showed it firing zero times across more than 17,000 real search nodes: propagation from the grid's pre-filled letters narrowed every domain below the branch's activation threshold within the first few assignments. All five were unreachable code the entire time we compared them, which is why their benchmark scores could not have distinguished them. The actual bottleneck was the small-domain branch's fixed word ordering; extending the solver's existing restart-randomization flag to it produced the improvements reported in Section V-B. The reusable point: an A/B comparison of branching heuristics is only informative once the branch under test is confirmed, by direct instrumentation, to execute.

V. Experiments

A. Setup

The experiments below test two distinct claims, and we keep them in separate subsections rather than blending them. Sections V-B–V-F test whether restart-portfolio search, as an architectural class, occupies a genuinely different point in the design space than partition-based search – using orca-solver as a concrete, independently developed representative of that alternative, and an unmodified naive backtracker to show the comparison is not trivial. This establishes why the mechanism is worth building at all. Section V-G then isolates the mechanism itself, holding architecture fixed, which is this paper's actual contribution.

Solvers. xfill (this work); orca-solver, a partition-based parallel solver treated as a black box; ingrid_core, a single-threaded reference solver; and, as a naive baseline with no restarts, no learned heuristics, and no parallelism, crossword-composer (a static-constraint-order Rust backtracker). A second naive baseline, savin_crossword, was tested but is excluded below: it timed out on all 20 grids at both a 120s and a 300s cap, including trivial 5×5/7×7 ones (Section V-D). The remaining four run through a single fixed harness (benchmarks/urtc2026_testbench/), against a dictionary derived deterministically from the repository's own freely available word list, with a fixed seed and a 300-second cap, reproducible via run_benchmark.py; every timeout was re-tested at 600s via rerun_timeouts.py to rule out an under-provisioned cap (Section V-D). xfill and orca-solver race independent workers, so wall time varies run to run; each runs 5 times per grid, median reported, every trial logged. ingrid_core and crossword-composer are single-threaded and deterministic, so each runs once.

Grid set. A size-graded curated set (5×5 through 21×21) plus a fixed-seed random sample of 15×15 grids scraped from a public corpus of real, previously published crossword layouts, crosswordgrids.com [10], all committed to the repository.

B. Restart Portfolios Exploit Added Parallelism

Restart-portfolio search should benefit from more independent workers even past the physical core count, because heavy-tailed runtime means each additional worker is another draw that might get lucky early. We tested this directly on a known-hard 7×7 grid (two pre-filled letters, no blocked cells, single sample), oversubscribing threads well past this machine's 14 physical cores: wall time falls from 166.7s at 1 thread to 63.0s at 14, then wanders (104.6s at 28, 85.6s at 42, 112.4s at 56) as contention erodes further gains. Every configuration beats the 201.7s single-threaded baseline an independently developed partition-based solver publishes on the same grid – an external difficulty anchor, not a target being chased.

C. Is This a Property of the Architecture, or One Grid?

Fig. 1 repeats the oversubscription experiment on three more grids, at 1 through 42 threads, for both xfill and orca-solver, to check whether Section V-B's result is a general property of restart portfolios or an artifact of one grid. The result splits three ways: on grid_120, xfill's wall time falls from 27.8s at 1 thread to 0.43s at 42 – a 65× improvement – while orca-solver never solves it below 14 threads and only creeps from 27.3s to 17.9s from 14 to 42; on grid_303, orca-solver fails to solve it at any of the 7 thread counts tested, while xfill solves it (noisily, not monotonically) at 8, 14, and 42 threads; on grid_115, neither solver succeeds at any thread count – this one favors neither architecture. Where the pattern does hold, it holds because absorbing extra parallelism without restructuring the search is what

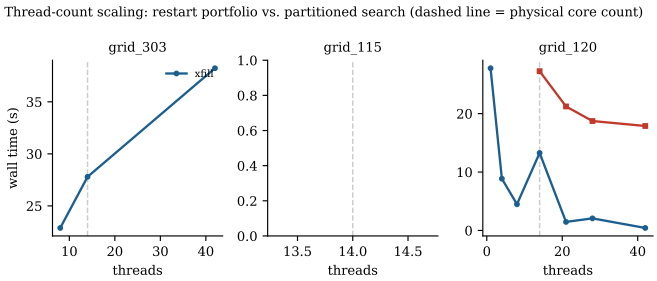

Fig. 1. Wall time vs. thread count, xfill vs. orca-solver, on three grids known to be hard for restart-based search (45s cap; orca-solver missing from a panel means every one of its 7 thread counts timed out). grid_120 is the clearest case: xfill improves roughly 65× from 1 to 42 threads while orca-solver stays flat and cannot solve at all below 14.

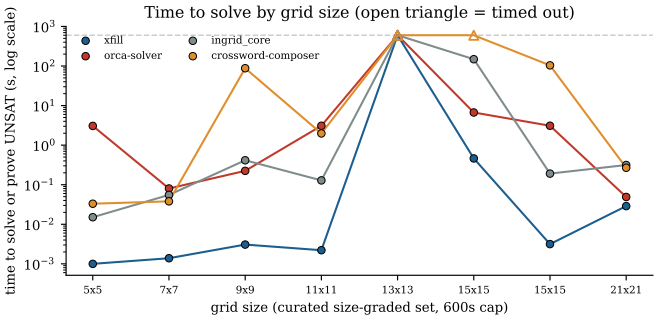

Fig. 2. Time to solve or prove UNSAT by grid size, curated set, 600s cap (log scale; open triangle = timed out). A solved/not-solved view of this same data would show xfill, orca-solver, and ingrid_core essentially tied, since all three succeed on the same grids here – the actual difference is that xfill's line sits one to three orders of magnitude below the others at every size except 13×13 (every solver times out together) and 21×21 (an easy UNSAT proof every solver resolves in well under a second regardless).

a non-partitioning restart portfolio is built to do; a partition-based design here does not exhibit that capacity to nearly the same degree.

D. Naive Backtracking Is Not a Baseline Worth Beating

Across all 20 grids, xfill resolved 19 (18 solved, 1 UNSAT; 1 timeout), orca-solver 18 (17 solved, 1 UNSAT; 2 timeouts), ingrid_core 17 (16 solved, 1 UNSAT; 3 timeouts), and crossword-composer 9 (8 solved, 1 UNSAT; 11 timeouts); savin_crossword resolved none, for the reasons given in Setup, and never appears in Fig. 2 or Fig. 3. Its failure differs in kind from crossword-composer's: crossword-composer solves most grids and fails only on individual instances, unrelated to dictionary size, while savin_crossword's least-constraining-value step recomputes an $O(|\text{domain}|^2)$ score from scratch at every node, which our ∼184,000-word dictionary makes prohibitive regardless of size – both real, unmodified, independently authored solvers, tested rather than assumed to fail. Doubling the cap to 600s changed exactly one outcome: ingrid_core went on to solve grid_479 in 325s.

Every other timeout – all 11 of crossword-composer's, plus sample_13x13, the one grid no solver resolves at either cap – held: evidence these are hard, not under-budgeted. The point for this paper's thesis is narrower than "xfill wins": at the grid and dictionary sizes crossword filling actually requires, restarts, adaptive heuristics, and parallelism are not optional refinements – they are the premise on which the rest of this section, and the mechanism this paper contributes, depend.

E. Aggregate Speed Advantage Where Both Solve

Fig. 3 quantifies the restart-portfolio/partition comparison on the scraped 15×15 corpus alone [10] – realistic-scale grids, rather than the curated size-graded set, which mixes in small grids that inflate the ratio. Of the 11 scraped grids (of 12) both xfill and orca-solver solve, xfill is faster on 9, with a median speedup of 494× and a geometric mean of 107×. Two exceptions run the other way: on grid_395 and grid_479, orca-solver is faster, by 5.4× and 1.6× – consistent with grid_115 in Section V-C, not every grid favors either architecture. grid_423, the twelfth scraped grid, sharpens this in the opposite direction: xfill solves it in 270s where orca-solver cannot, even given double the budget – a completion advantage, not just a speed one. Across the full 20-grid sample, xfill resolves 19, orca-solver 18, both agree on the sample's one correct UNSAT proof, and both are stuck only on sample_13x13 – a narrower gap on success alone than on speed, which is why Sections V-C and V-G use a harder, pre-existing grid list to find where the architectures actually diverge.

F. Exhaustive Case Study: Proving, Not Just Finding

Every experiment so far tests finding a solution quickly; partitioning's genuine non-overlapping coverage (Section II) should instead matter for proving none exists. To test this, we constructed by hand a maximally constrained grid – five mutually crossing 15-letter entries, verified against both solvers' parsers – and ran both to completion with no time limit. Both independently concluded no solution exists: xfill exhausted 2.16 billion nodes over 791 restarts in 76 minutes; orca-solver reported search exhausted after enumerating 224 partitions in 25 minutes, roughly 3× faster. This is the one result where partitioning, not the restart portfolio, has the structural advantage – consistent with Section II.

G. Isolating the Mechanism: A Shared-Weight Ablation

Everything above tests xfill's architecture against a different codebase's; this section tests xfill against itself, with one environment variable flipped: same solver, same grid, same dictionary, same 14 threads, XFILL_DISABLE_SHARED_WEIGHTS on and off, several trials each, on grids from two independent sources: the known-hard list used throughout Section V, and three grids from the paper's own main scraped-15×15 corpus, to check the first result is not an artifact of one curated list.

Fig. 4 does not show a uniform win, on either grid set, and we report that plainly. On grid_053, the unshared baseline is faster on the median (0.72s vs. 2.77s) and far tighter (0.66–0.76s vs. 0.07–3.71s). On grid_058 and grid_479, medians are similar but shared weights show wider spread. On grid_120, shared weights reach a lower best case but also produce the only outright timeout among those eighteen trials. grid_395 and grid_423 sharpen both directions: a clean 7–10× win on the former (36.9s vs. 271.4s median), and actively worse on the latter – all 5 trials hit the 300s cap, where disabling shared weights solves it reliably at ∼180s every time. That last result complicates Section V-E: the same run that gave xfill its completion advantage over orca-solver on grid_423 also timed out in 2 of its own 5 trials – shared weights are exactly what make that grid unreliable for xfill in the first place. Against Section III's broader-corpus finding of a consistent win, the synthesis is regime-dependent: shared weights help on typical grids by redistributing which crossings each worker deprioritizes, but at the extreme difficulty end they trade tight, well-behaved variance for a wider one that is not reliably better, and sometimes reliably worse. We have no evidence for a uniform win, so we do not claim one – the same standard Section IV applies to the heuristics it rules out.

VI. Discussion

The architectural comparison and the mechanism ablation share the same underlying cause. Restart portfolios benefit from heavy-tailed runtimes: more independent draws raise the chance of an early lucky one, which is why oversubscribing thread count helped in Section V-B and Fig. 1, and why grid_423 (Fig. 3) goes further: more draws can succeed where a fixed partition cannot. Partitioned search instead gives genuine non-overlapping coverage, which is what Section V-F's proof of unsatisfiability required. The same heavy-tailedness explains Fig. 4's mixed result: shared conflict weights change which draw from the tail each worker takes, which can land on a worse draw as easily as a better one, netting out as an improvement only once averaged over a broader, typical corpus.

VII. Limitations and Conclusion

All timing results are from one 14-core machine and one dictionary derived from a specific freely available word list; neither the thread-scaling numbers nor Section V-F's unsatisfiability result should be read as hardware- or dictionary-independent claims. Both principal solvers are actively developed; exact commit hashes are recorded in the testbench README.

This paper's contribution is the mechanism in Section III: concurrent conflict-weight sharing across a restart portfolio that never partitions, occupying a gap that three adjacent lines of prior work leave open. The architectural comparison in Section V supports that gap's importance

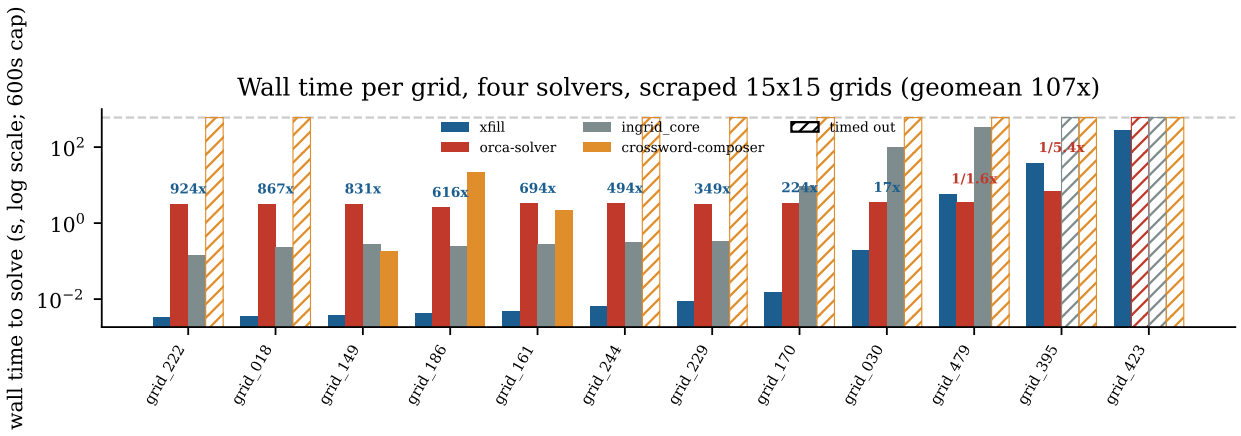

Fig. 3. Wall time to solve (log scale, 600s cap), four solvers, per scraped 15×15 grid, sorted by xfill's time. Hatched bars mark a timeout; crossword-composer times out on most grids, the same pattern reported by count in Section V-D (savin_crossword is excluded entirely, also Section V-D). Label above the xfill/orca-solver pair is orca-solver time / xfill time; two grids (grid_395, grid_479) favor orca-solver, and on grid_423 orca-solver cannot solve it even at this cap while xfill does, in 270s.

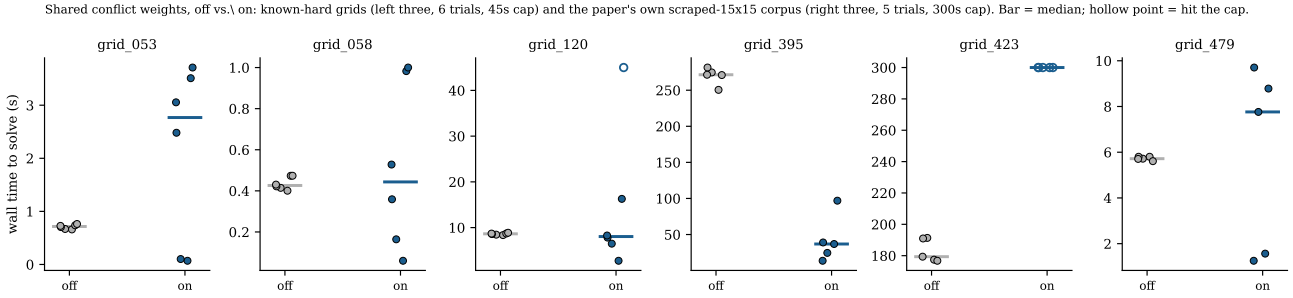

Fig. 4. Shared-conflict-weight ablation, off vs. on, 14 threads. Left three panels: the known-hard list from Fig. 1, 6 trials each, 45s cap. Right three: three grids from the paper's own scraped-15×15 corpus (the other 9 of 12 solve in well under a second regardless of the toggle and are omitted as uninformative), 5 trials each, 300s cap. Bars mark the median; hollow points hit the cap; the spread is the finding.

rather than being the paper's subject – restart portfolios and partitioned search are suited respectively to finding solutions quickly and to proving none exist, a pattern independent of any one solver. The mechanism itself, tested in isolation, is more modest than the architecture it sits inside: a real win on typical grids, a wash or worse on the hardest ones, reported as measured rather than as hoped. Nothing about it is specific to crossword filling: it needs only a wdeg-style weight table and a restart loop, both already common in CSP and lazy-clause-generation solvers. We also reported a negative result from the same codebase – five heuristics rendered indistinguishable by a precondition that kept them from ever executing – as a reusable caution for comparing branching heuristics by benchmark score alone.

Acknowledgment

[Advisor/funding acknowledgments.]

References

[1] M. L. Ginsberg, "Dr.Fill: crosswords and an implemented solver for singly weighted CSPs," Journal of Artificial Intelligence Research, vol. 42, pp. 851–886, 2011.

[2] C. P. Gomes, B. Selman, and H. Kautz, "Boosting combinatorial search through randomization," in Proc. AAAI, 1998, pp. 431–437.

[3] C. P. Gomes, B. Selman, N. Crato, and H. Kautz, "Heavy-tailed phenomena in satisfiability and constraint satisfaction problems," Journal of Automated Reasoning, vol. 24, pp. 67–100, 2000.

[4] F. Boussemart, F. Hemery, C. Lecoutre, and L. Sais, "Boosting systematic search by weighting constraints," in Proc. ECAI, 2004, pp. 146–150.

[5] D. Grimes and R. J. Wallace, "Sampling strategies and variable selection in weighted degree heuristics," in Proc. CP, LNCS vol. 4741, 2007, pp. 831–838.

[6] Y. Hamadi, S. Jabbour, and L. Sais, "ManySAT: a parallel SAT solver," J. Satisfiability, Boolean Modeling and Computation, vol. 6, pp. 245–262, 2009.

[7] T. Ehlers and P. J. Stuckey, "Parallelizing constraint programming with learning," in Proc. CPAIOR, LNCS vol. 9676, Banff, Canada, 2016.

[8] X. Yun and S. L. Epstein, "A hybrid paradigm for adaptive parallel search," in Proc. CP, LNCS vol. 7514, 2012, pp. 720–736.

[9] I. P. Gent, I. J. Miguel, P. W. Nightingale, C. McCreesh, P. Prosser, N. Moore, and C. Unsworth, "A review of literature on parallel constraint solving," Theory and Practice of Logic Programming, vol. 18, no. 5–6, pp. 725–758, 2018.

[10] Crosswordgrids.com, crosswordgrids.com/15x15-crossword-grids